临界区与锁 – 扩展

中国科学院大学计算机学院
2026-09-28

中国科学院计算技术研究所 INSTITUTE OF COMPUTING TECHNOLOGY, CHINESE ACADEMY OF SCIENCES



常见的锁类型

- 自旋锁
  - 一个线程试图获取锁时，如果获取不到，则会不断“自旋”，即循环检查锁的状态，直到锁可用
  - 适用于锁定时间非常短的场景，开销小
- 互斥锁
  - 一个线程试图获取锁时，如果获取不到，则会进入等待状态
  - 当线程释放锁时，根据调度策略决定哪个线程获得锁，例如按锁的请求顺序决定获得锁的线程



常见的锁类型

- 读者-写者问题
  - 多个并发线程/进程共享一份数据
  - 读者：只读取数据
  - 写者：修改数据
  - 典型场景
    - 内存中的共享配置表、缓存中的共享对象、文件系统的文件元数据
- 要求
  - 读读允许：允许多个读者同时访问共享资源
  - 写写互斥：写者在进行写操作时要独占资源
  - 读写互斥：不允许读者和写者同时访问资源
- 不同策略
  - 读者优先：只要有一个读者在读，后续读者可以继续进入，即使有写者在等待
  - 写者优先：一旦有写者在等待，新的读者不能再进入，直到写者完成

常见的锁类型

- 读写锁
  - 读操作
    - 如果没有写者持有锁，且没有任何写者正在等待获取锁（写者优先），那么读者可以获得读锁并访问共享资源
    - 如果没有写者持有锁，那么读者可以获得读锁并访问共享资源（读者优先）
  - 写操作
    - 如果没有写者持有锁，且所有持有的读锁都已释放，则可以获得写锁



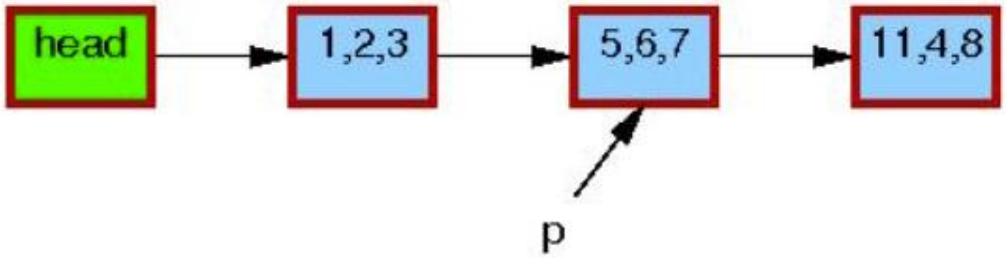
常见的锁类型

- RCU机制
  - 读写锁不足
    - 读操作需要获取锁：原子增加读者计数；检查是否有写者持有锁或在等待
    - 读者很多时，写者延迟增加甚至饿死（读者优先）
    - 写者等待时会阻塞新的读者，导致读吞吐下降（写者优先）
  - Read-Copy-Update
    - 读操作不需要获取锁，可以并发执行
    - 写操作首先复制旧数据，在副本上进行更新
    - 更新完成后，使用RCU函数更新指针（rcu_assign_pointer）或替换节点（list_replace_rcu）
    - 确认所有当前的读操作已完成，再将旧数据清理（释放内存）
  - 适用于高并发读（读多写少）且读性能要求极高的场景
    - 路由表查询
    - 访存查询页表
    - 安全策略查询
    - 文件系统元数据查询

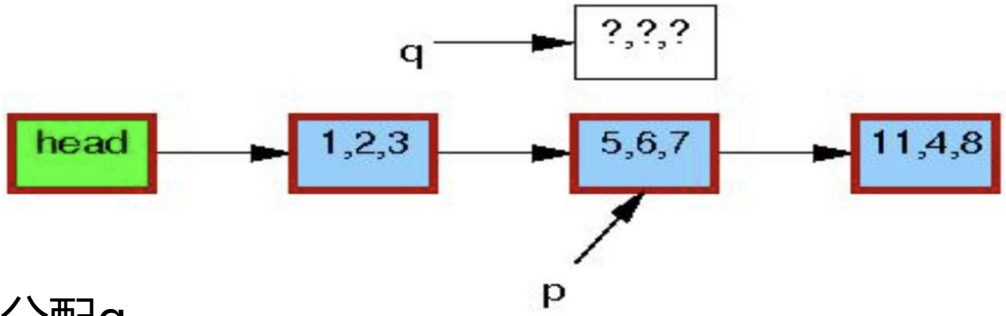
常见的锁类型

- RCU示例

```
1 q = kmalloc(sizeof(*p), GFP_KERNEL);
2 *q = *p;
3 q->b = 2;
4 q->c = 3;
5 list_replace_rcu(&p->list, &q->list);
6 synchronize_rcu();
7 kfree(p);
```



初始状态

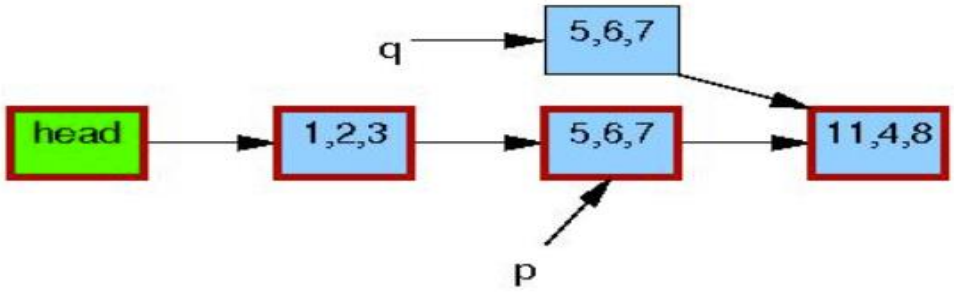


分配q

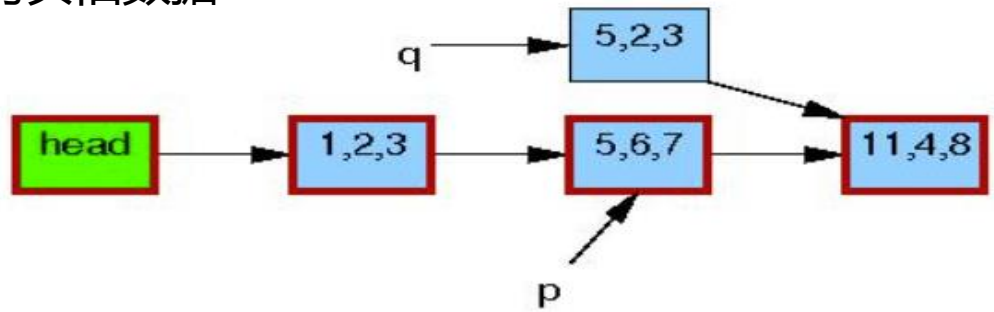
常见的锁类型

- RCU示例

```
1 q = kmalloc(sizeof(*p), GFP_KERNEL);
2 *q = *p;
3 q->b = 2;
4 q->c = 3;
5 list_replace_rcu(&p->list, &q->list);
6 synchronize_rcu();
7 kfree(p);
```



拷贝旧数据

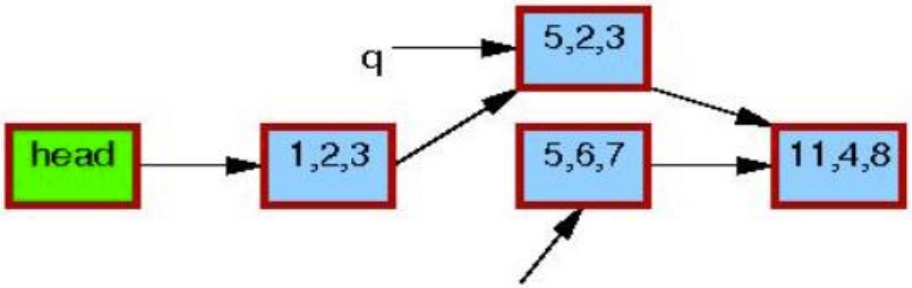


更新数值

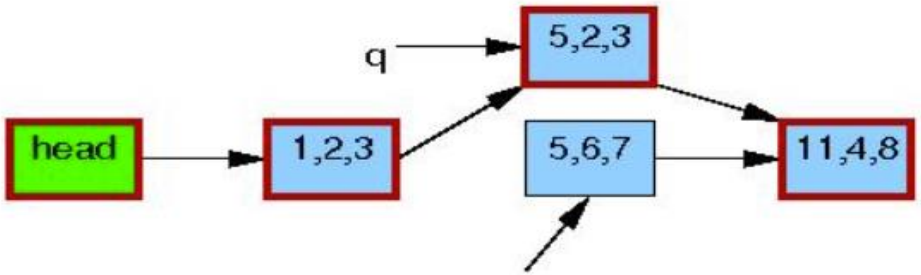
常见的锁类型

- RCU示例

```
1 q = kmalloc(sizeof(*p), GFP_KERNEL);
2 *q = *p;
3 q->b = 2;
4 q->c = 3;
5 list_replace_rcu(&p->list, &q->list);
6 synchronize_rcu();
7 kfree(p);
```



更新链表

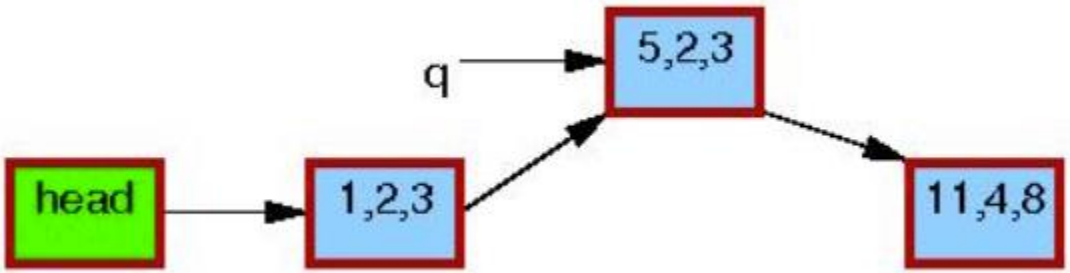


等待当前所有读者完成

常见的锁类型

- RCU示例

```
1 q = kmalloc(sizeof(*p), GFP_KERNEL);
2 *q = *p;
3 q->b = 2;
4 q->c = 3;
5 list_replace_rcu(&p->list, &q->list);
6 synchronize_rcu();
7 kfree(p);
```



链表完成更新，旧数据被释放